



PROGRAMACIÓN ORIENTADA A OBJETOS

TRABAJO PRÁCTICO N°5

1. ¿Que es un patrón de diseño?
2. Investigue e identifique las diferentes categorías de patrones de diseño que existen.
3. Explique cómo funciona el patrón Singleton para establecer la conexión con una base de datos de Postgresql o MySql.
4. Desarrollar mediante el patrón Builder tres tipos de configuraciones para Personajes:
 - a. Guerrero, equipado con Espada Corta y Armadura de cuero.
 - b. Mago, equipado con Báculo y Túnica de seda.
 - c. Mecha, equipado con Cañón de plasma y Exoesqueleto de titanio.

El producto final obtenido debe poseer el equipamiento y el chasis/cuerpo como sus componentes y atributos principales.

5. Implementar el patrón Factory Method con el fin de crear instancias de juegos tanto físicos como digitales. Ambos tipos de juegos deben compartir propiedades comunes: un identificador (id) y un importe base. Asimismo, se debe definir el método abstracto getPrecio para determinar el costo final según corresponda. En el caso de los juegos físicos, se añadirá una propiedad float que representa el costo de envío (establezca un valor de ejemplo a su elección). Por otro lado, para los juegos digitales, el costo estará sujeto a la plataforma de adquisición, incluyendo un atributo float para el precio de la misma. Para obtener el importe final, se deberán multiplicar dichos valores por el importe base.
6. Aplicar el patrón de diseño Facade mediante la creación de un subsistema de localización. Para ello, desarrolle la clase TraductorHelper, la cual simulará la traducción de términos de español a inglés a través del método traducirEspanolAlIngles(String palabra), utilizando un diccionario con al menos 5 palabras asociadas a videojuegos. Asimismo, construya la clase FormateadorTextoHelper que exponga dos métodos: aMayusculas(String texto) y aMinusculas(String texto), encargados de transformar el texto ingresado al formato respectivo. Por último, establezca la clase LocalizacionFacade para que sirva como punto de acceso centralizado, instanciando ambos componentes internamente y publicando sus operaciones con los siguientes tres métodos:
 - a. traducir(String palabra)
 - b. convertirAmayusculas(String texto)
 - c. convertirAminusculas(String texto)



PROGRAMACIÓN ORIENTADA A OBJETOS

TRABAJO PRÁCTICO N°5

7. Aplique el patrón Composite para modelar el Inventario de un Personaje, de manera tal que pueda contener tanto Items individuales como Bolsas. Una Bolsa contiene items u otras Bolsas. Defina la estructura que considere pertinent y llámela ElementoInventario con el método `getPesoTotal()`, de manera que tanto un Item como una Bolsa respondan a este método de forma uniforme para calcular el peso de las Bolsas.
8. Aplique el patrón Decorator para modificar dinámicamente las estadísticas de un Arma base. Defina la clase Arma con un daño base, y al menos dos decoradores (por ejemplo, `ArmaConFuego` y `ArmaConVeneno`) que envuelvan un arma existente y le agreguen un bonus de daño adicional al invocar `getDanio()`. Combine ambos decoradores sobre una misma arma y muestre por consola el daño resultante.
9. Retome la `VentanaJuego` desarrollada en el TP4 y aplique el patrón Observer para hacer que `Heroe` notifique a sus observadores cada vez que cambian sus puntos de vida, y así actualizar el componente visual correspondiente. Implemente al menos dos observadores concretos: `BarraDeVidaObserver`, que reciba la referencia al `JProgressBar` de vida y actualice su valor cada vez que sea notificado, y `RegistroCombateObserver`, que anexe una línea del evento a un `JTextArea` de log dentro de la misma ventana.
10. Aplique el patrón State para modelar el comportamiento de un Guardia en un juego de sigilo, con estados como `Patrullando`, `EnAlerta` y `Persiguiendo`, de manera que el método `actuar()` varíe según el estado actual: en el estado `Patrullando` el guardia se desplaza entre puntos fijos, en estado `EnAlerta` permanece quieto investigando un ruido, y en el estado `Persiguiendo` avanza hacia la posición del jugador. Simule, dentro del `main`, la transición del guardia entre al menos dos de estos estados. Para la representación visual alcanza con un rectángulo de color por personaje sobre un `JPanel`.
11. Aplique el patrón Strategy para implementar distintos comportamientos en un juego de Ta-Te-Ti. Defina una interfaz `EstrategiaJuego` con el método `elegirMovimiento(tablero)` y al menos dos implementaciones (por ejemplo, `EstrategiaAleatoria`, que elija una casilla libre al azar. Simule una partida donde la CPU cambie de estrategia entre turnos. Puede representar el tablero en la consola cómo una matriz de texto de 3x3.
12. Investigue y aplique el patrón DAO (data access object) a un sistema que usted elija para conectarlo con la base de datos.